Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251172**
Nama Lengkap	**Ang, Joshua Alexander Hartono**
Minggu ke / Materi	**07 / Percabangan dan Perulangan Kompleks**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

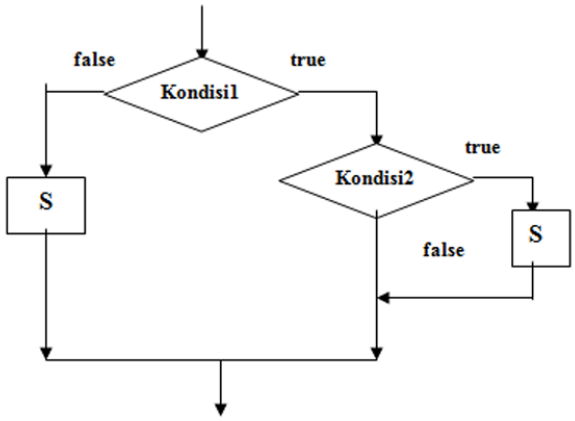
Struktur Percabangan Kompleks

Sutruktur percabangan kompleks merupakan pengembangan dari percabangan biasa yang memungkinkan program untuk menangani lebih dari satu kondisi. Dalam percabangan ini, program dapat mengecek beberapa kondisi yang ada secara berurutan. Struktur ini biasanya menggunakan kombinasi perintah IF, else, dan percabangan bertigkat seperti IF dalam IF.

Dalam percabangan kompleks terdapat beberapa bentuk variasi susunan kondisi. Setiap bentuk memiliki alur yang berbeda tetapi memiliki satu tujuan yang sama yaitu menentukan aksi jika suatu kondisi terpenuhi. Contohnya pada percabangan kompleks pertama, program akan mengecek kondisi utama terlebih dahulu. Jika kondisi utama terpenuhi maka program akan mengecek kondisi berikutnya. Jika tidak terpenuhi, maka program akan langsung menjalankan bagian else.

```
if kondisi1:
    if kondisi2:
        S
        S
else:
    S
    S
```

Gambar 7.1 dan gambar 7.2: Contoh pecabangan kompleks bentuk 1. *Sumber:* https://l1nq.com/OuRVv



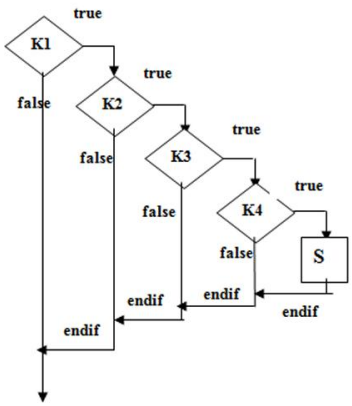
Gambar 7.3: Gambar flowchart percabangan kompleks bentuk 1. *Sumber:* https://l1nq.com/OuRVv

Selanjutnya terdapat sturktur pecabangan kedua, struktur yang digunakan merupakan percabangan IF yang berada dalam IF. Dalam hal ini program akan melakukan pengecekan kondisi secara berurutan dari kondisi pertama hingga kondisi berikutnya. Jika semua kondisi terpenuhi maka perintah percabangan paling akhir akan dijalankan.

```
1 if kondisi1:
2     if kondisi2:
3         if kondisi3:
4             if kondisi4:
5                 S
```

Gambar 7.4 dan gambar 7.5: Contoh pecabangan kompleks bentuk 2. *Sumber:* https://l1nq.com/OuRVv

Pada sturkur tersebut, setiap kondisi saling bergantung satu sama lain. Sehingga kondisi kedua akan terperiksa jika kondisi pertama terpenuhi, kondisi ketiga akan diperiksa jika kondisi kedua terpenuhi, dan seterusnya. Bentuk ini biasanya membutuhkan pengecekan kondisi secara bertahap di mana setiap langkah harus benar agar dapat melanjutkan pengecekan pada tahap berikutnya.



Gambar 7.6: Gambar flowchart percabangan kompleks bentuk 2. *Sumber:* https://l1nq.com/OuRVv

Selain bentuk pecabangan tersebut, terdapat perbedaan antara penggunaan IF secara terpisah dengan penggunaan IF bertingkat menggunakan ELIF. Pada penggunaan IF biasa, semua kondisi akan dicek satu persatu walau kondisi yang memenuhi sudah tercapai. Hal ini menyebabkan proses menjadi kurang efisien karena program tetap mengevaluasi seluruh kondisi yang ada.

```
if (kondisi1):
    instruksi1
if(kondisi2):
    instruksi2
if(kondisi3):
    instruksi3
if(kondisi4):
    instruksi4
```

Gambar 7.7: Sturktur IF bertingkat biasa. *Sumber:* https://l1nq.com/OuRVv

Berbeda dengan penggunaan IF bertingkat yang menggunakan ELIF, di mana program akan otomatis berhenti mengecek kondisi jika suatu kondisi sudah terpenuhi. Sehingga proses eksekusi program menjadi lebih cepat dan efisien.

```
if (kondisi1):
    instruksi1
elif(kondisi2):
    instruksi2
elif(kondisi3):
    instruksi3
elif(kondisi4):
    instruksi4
```

Gambar 7.8: Sturktur IF bertingkat menggunakan ELIF. *Sumber:* https://l1nq.com/OuRVv

StrukturPerulanganKompleks

Perintah break digunakan untuk memberhentikan perulangan pada program. Biasanya perintah ini digunakan ketika suatu kondisi tertentu sudah terpenuhi sehingga perulangan tidak perlu dilanjutkan. Break sendiri ditandai dengan penggunan perintah IF.

```
for i in range(1000):
    if i==10:
        break
    print(i)
```

Gambar 7.9: Contoh penggunaan break. *Sumber:* https://l1nq.com/OuRVv

Pada contoh tersebut, perulangan akan berhenti jika nilai i mencapai nilai 10 meskipun trerdapat perulangan sampai dengan 1000. Break dalam hal ini berfungsi mengontrol perulangan agar lebih efisien.

Output:

```
0
1
2
3
4
5
6
7
8
9
```

Gambar 7.10: Output penggunaan break pada gambar 7.9. *Sumber:* https://l1nq.com/OuRVv

Selain break, terdapat pula perintah continue yang berfugnsi untuk melewati satu iterasi jika kondisi tertentu tercapai dan kemudian dilanjutkan ke iterasi selanjutnya.

```
for i in range(10):
    if i==5:
        continue
    print(i)
```

Gambar 7.11: Contoh penggunaan continue. *Sumber:* https://l1nq.com/OuRVv

Output:

```
0
1
2
3
4
6
7
8
9
```

Gambar 7.12: Output penggunaan continue pada gambar 7.11. *Sumber:* https://l1nq.com/OuRVv

Terahkir terdapat perulangan bertingkat atau sering disebut nested loop. Nested loop adalah sebuah struktur perulangan di mana terdapat perulangan dalam sebuah perulangan. Nested loop sendiri biasa digunakan untuk menyelesaikan malasah yang berkaitan dengan bentuk dua dimensi, catur, minesweeper, atau pengolahan citra digital.

```
1 for i in range(m):
2     for j in range(n):
3         <lakukan perintah ini di inner>
4         <lakukan perintah itu di inner>
5     <lakukan perintah lain di outer>
6     <lakukan perintah lain lagi di outer>
```

Gambar 7.13: Struktur dalam nested loop. *Sumber:* https://l1nq.com/OuRVv

Pada contoh tersebut, perulangan menggunankan i disebut dengan outer loop dan j sebagai inner loop. Setiap kali outer loop dijalankan inner loop akan dijalankan sebanyak n kali.

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : https://github.com/angjoshua-sudo/Prak-Alpro-2026.git

SOAL 1

Source code:

```python
n = int(input("masukkan bilangan yang ingin dicari prima terdekatnya: "))

if n <= 2:
    print(f"tidak ada bilangan prima yang lebih kecil dari: {n}")
else:
    for i in range(n-1, 1, -1):
        for j in range(2, i):
            if i % j == 0:
                break
        else:
            print(f"input n={n}, maka prima terdekat < {n} adalah {i}")
            break

```

Penjelasan:

Program ini di awali dengan meminta user untuk memberikan input angka untuk dicari angka prima terdekat sebelum angka yang diinput oleh user. Input ini akan disimpan dalam variabel **n**. setelahnya, program akan mengecek apakah n kurang dari atau sama dengan 2. Jika iya maka program akan memberikan output "**tidak ada bilangan prima yang lebih kecil dari: n**". Jika tidak, program akan melanjutkan ke bagian else.

Bagian else di awali dengan **perulangan i** yang dimulai dari **n-1 sampai dengan angka 2** dengan **step -1**. Dalam setiap perulangan i, akan dicek apakah angka tersebut prima atau tidak. Jika angka yang dicek habis dibagi dengan j maka angka tersebut bukanlah angka prima dan akan menjalankan perintah **break** kemudian dilanjutkan ke-i selanjutnya. Namun, jika tidak ada j yang dapat membagi habis i, maka program akan lanjut ke bagian else dan memberikan output "**input n=n, maka prima terdekat < n adalah i**" yang kemudian dilanjutkan dengan **break** untuk menghentikan perulangan dari mencari angka prima berikutnya.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu7/soal1.
py
masukkan bilangan yang ingin dicari prima terdekatnya: 21
input n=21, maka prima terdekat < 21 adalah 19
```

SOAL 2

Source code:

```
minggu7 > soal2.py > ...
n = int(input("masukkan nilai n: "))
print(f"n = {n}")

for i in range(n, 0, -1):
    hasil = 1
    for j in range(i, 0, -1):
        hasil *=  j
    print(hasil, end=" ")

    for k in range(i, 0, -1):
        print(k, end=" ")
    print()
```

Penjelasan:

Program ini dimulai dengan meminta nilai **n** dari pengguna menggunakan fungsi input. Setelahnya, nilai **n** akan dicetak oleh program menggunakan fungsi **print**. Kemudian program akan masuk ke perulangan **for i in range(n, 0, -1**) untuk megatur jumlah baris yang akan dikeluarkan.

Dalam perulangan terdapat variabel **hasil** dengan nilai 1 pada setiap iterasi. Variabel ini digunakan untuk menghitung nilai dari perkalian (faktorial) dari nilai **i** pada setiap baris. Perhitungan kemudian dilakukan pada loop kedua yaitu **for j in range(i, 0, -1)**. loop ini akan menghasilkan nilai faktorial yang kemudian dicetak menggunakan **print(hasil, end=" ")** agar tidak pindah ke baris baru. Setelahnya program akan menjalankan loop ketiga **for k in range(i, 0, -1).** Loop ini berfungsi untuk menampilkan angka dari i sampai dengan 1. Kemudian hasil akan dicetak dalam satu baris bersama dengan faktorial sebelumnya. Kemudian program akan menjalankan bagian **print()** untuk berpindah ke baris selanjutnya. Kemudian program akan melakukan perulangan i sampai nilainya sama dengan 1.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu7/soal2.py
masukkan nilai n: 6
n = 6
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

SOAL 3

Source code:

```python
tinggi = int(input('masukkan tinggi: '))
lebar = int(input('masukkan lebar: '))

print(f"tinggi = {tinggi}, lebar = {lebar}")

l = tinggi * lebar
awal = 1
akhir = lebar
for i in range(1, tinggi+1):
    for j in range(awal, akhir+1):
        print(j, end=" ")
    print()
    awal += lebar
    akhir += lebar
```

Penjelasan:

Program dimulai dengan meminta input pada pengguna berupa nilai **tinggi** dan **lebar**. Kemudian kedua variabel tersebut dicetak menggunakan fitur print. Setelah itu, terdapat variabel **l** untuk menentukan batasan angka yang ditampilkan. Kemudian ada variabel **akhir** dan **awal** untuk menentukan awal dam akhir pada setiap baris dalam perulangan.

Selanjutnya, program akan menggunakan perulangan **for i in range(1, tinggi+1)** untuk mengatur jumlah baris yang akan dikeluarkan sesuai dengan jumlah tingggi. Di dalamnya ada loop kedua yaitu **for j in range(awal, akhir+1)** yang berfungsi untuk mencetak angka secara berurutan dalam satu baris. Kemudian nilai **j** akan terus bertambah sesuai dengan variabel **awal** dan **akhir**. Setelah satu baris selesai, maka program akan lanjut menjalankan **print()** untuk pindah ke baris berikutnya. Setelah mencetak satu baris maka program akan menambahkan nilai **awal** dan **akhir** dengan **lebar**. Hal ini bertujuan agar baris berikutnya dapat melanjutkan angka dari baris sebelumnya.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu7/soal3.
py
masukkan tinggi: 5
masukkan lebar: 4
tinggi = 5, lebar = 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github>
```